

Interactive User Environment Application on Kubernetes Platform

Chalvatzis Kyriakos



TUC

July 10, 2025



Supervisor: Prof. Vasilis Samoladas
Committee: Prof. Nikos Giatrakos
Prof. Euripides Petrakis



Outline

- 1 Introduction
- 2 Objectives
- 3 Methodology
- 4 Results
- 5 Conclusion



- Data analytics has become essential across all fields — but entry-level users often struggle with the required infrastructure.
- Motivation: Lower the barrier for non-experts to explore and process data using standard tools. User-friendly analytical computing!
- While systems like Apache Spark, Airflow, and Hadoop are powerful, they are built for experienced engineers.
- No current open system provides isolated, user-scoped, web-accessible environments on Kubernetes — tailored for batch analytics.



- Although not a research thesis, this work aims to prototype a full-stack system that provides:
 - Secure, multi-user access
 - Data upload and sharing capabilities
 - Batch job execution in a Kubernetes-native way
 - Integrated application-like tools such as DuckDB, Python Pandas, Octave and more can be applied on datasets
- Goal: Deliver an introductory platform for data analytics without overwhelming users.
- Goal: Achieve a Cloud-native and modular design.



- Microservice-based architecture deployed on Kubernetes
 - Why? "Seperation of Concerns", scalable, modular and adaptable
- Core components:
 - **Uspace API**: User-facing logic for datasets, jobs, resources
 - **FsLite**: Virtual filesystem abstraction with permissions
 - **MinIO**: Object storage backend
 - **Minioth**: Authentication and group-based access control
 - **FrontApp**: Web frontend (HTML templates, HTMX, WebSocket integration)
- Technologies: Go, Gin, Kubernetes, Docker

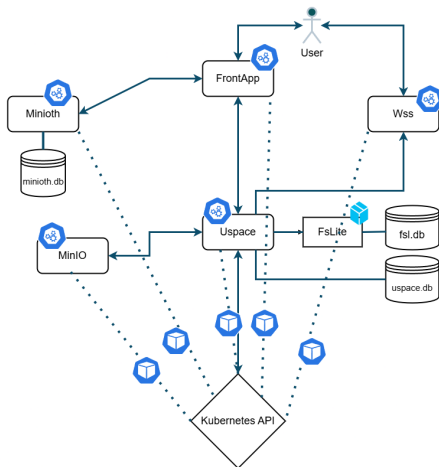


What we want to achieve in this system:

- Identity - (Authentication)
- Secure Access - (Authorization)
- Physical Storage - (Data Storage)
- Job execution mechanism - (Job Flow)



System Diagram



Uspace Storage Control

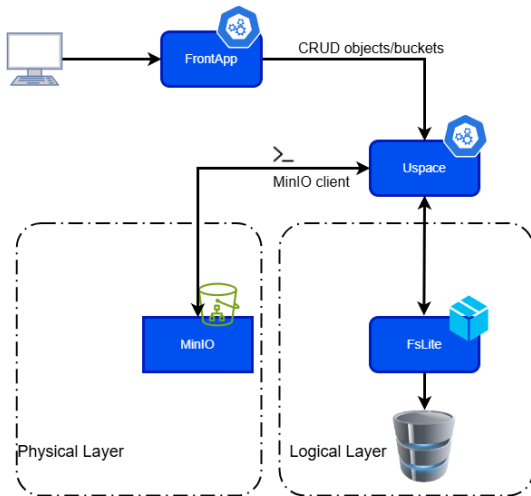


Figure: UspaceStorage



Uspace Job Lifecycle

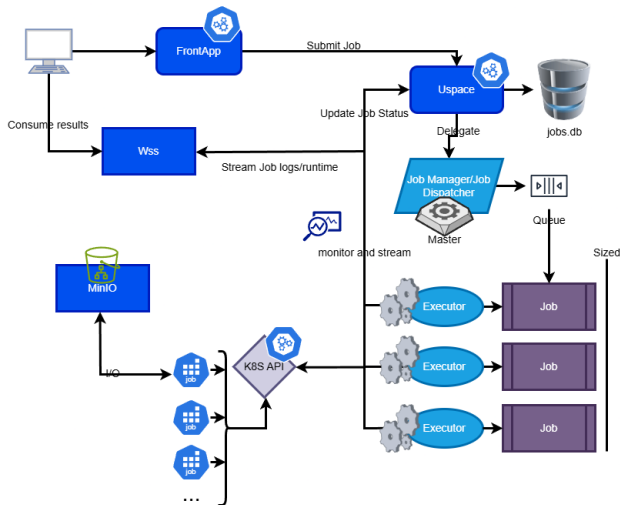
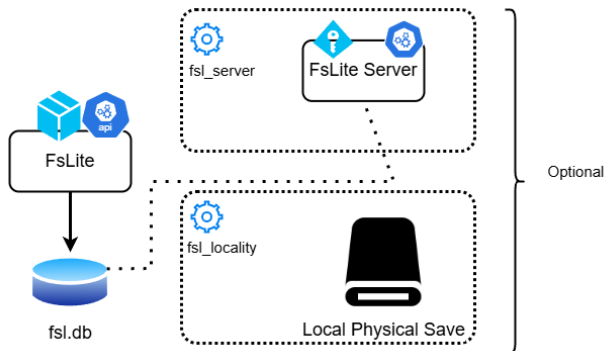


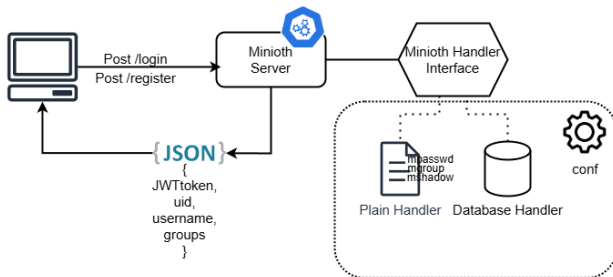
Figure: Uspace Jobs



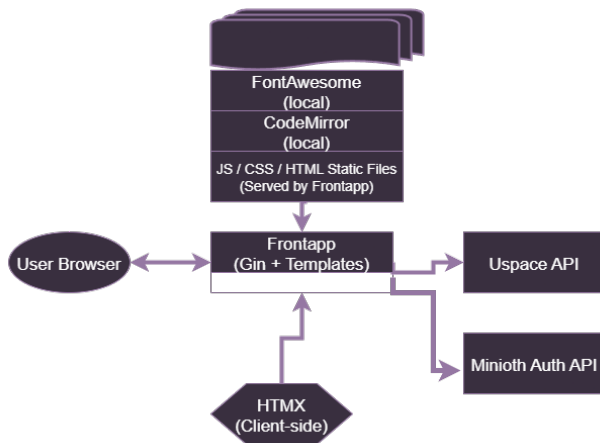
FsLite Structure



Minioth Structure



Frontapp Structure



Uspace API 1

Method	Path	Description	Requires Access-Target
GET	/healthz	Health check endpoint.	No
GET/POST	/api/v1/job	Submit or list jobs.	No
GET/POST	/api/v1/app	List available tools (applications).	No
GET	/api/v1/resources	List resources ("ls" equivalent).	Yes
POST	/api/v1/resource/upload	Upload a new resource file.	Yes
GET	/api/v1/resource/preview	Preview a resource.	Yes (Read)
GET	/api/v1/resource/download	Download a resource.	Yes (Read)
DELETE	/api/v1/resource/rm	Delete a resource.	Yes (Write)
POST	/api/v1/resource/cp	Copy a resource.	Yes (Read)
PATCH	/api/v1/resource/mv	Move/rename a resource.	Yes (Write)
PATCH	/api/v1/resource/permissions	Change resource permissions.	Yes (Owner)
PATCH	/api/v1/resource/ownership	Change resource ownership.	Yes (Owner)
PATCH	/api/v1/resource/group	Change associated group.	Yes (Owner)

Table: Uspace User API Endpoints



Uspace API 2

Method	Path	Description
GET/POST/PUT/DELETE/PATCH	/api/v1/admin/volumes	Full volume management API.
DELETE/PUT	/api/v1/admin/job	Administrative job control (e.g., delete jobs).
GET/POST/PATCH/DELETE	/api/v1/admin/user/volume	Manage user-volume mappings.
GET/POST/PUT/DELETE	/api/v1/admin/app	Manage available applications/tools.
GET	/api/v1/admin/system-conf	View current system configuration.
GET	/api/v1/admin/system-metrics	System metrics via Kubernetes API.

Table: Uspace Admin API Endpoints



Method	Path	Description
POST	/login	Authenticate and receive a token.
POST	/admin/register	Authenticate and receive JWT token.
POST	/admin/volume/new	Change password for the authenticated user.
DELETE	/admin/volume/delete	Delete a specified volume
GET	/admin/volume/get	Retrieve information about a logical volume
GET	/admin/resource/get	Retrieve metadata of multiple resources
GET	/admin/resource/stat	Retrieve more detailed metadata of a specific resource.
DELETE	/admin/resource/delete	Delete a specified resource.
POST	/admin/resource/upload	Upload resource content to volume.
GET	/admin/resource/download	Download a resource from storage.
GET/PATCH/DELETE	/admin/user/volumes	CRUD user volumes.

Table: FsLite public API endpoints



Minioth API 1

Method	Path	Description	Auth
POST	/v1/register	Register a new user.	No
POST	/v1/login	Authenticate and receive JWT token.	No
POST	/v1/passwd	Change password for the authenticated user.	Yes (user/admin)
GET	/v1/user/me	Get information about the token's user.	Token
GET	/v1/user/token	View current token details.	No
POST	/v1/user/refresh-token	Request a new access token using a refresh token.	No
GET	/v1/swagger/*any	Swagger UI for live API documentation.	No

Table: Minioth public/user API endpoints



Minioth API 2

Method	Path	Description
GET	/v1/admin/audit/logs	Retrieve system audit logs.
POST	/v1/admin/haser	Generate bcrypt hashes from plaintext passwords.
POST	/v1/admin/verify-password	Compare a password against a stored hash.
GET	/v1/admin/users	List all registered users.
GET	/v1/admin/groups	List all groups and their members.
POST	/v1/admin/useradd	Add a new user to the system.
DELETE	/v1/admin/userdel	Delete a user by UID or username.
PATCH	/v1/admin/userpatch	Update selected fields of a user.
PUT	/v1/admin/usermod	Fully replace a user record.
POST	/v1/admin/groupadd	Create a new group.
PATCH	/v1/admin/grouppatch	Update specific fields of a group.
PUT	/v1/admin/groupmod	Fully update a group definition.
DELETE	/v1/admin/groupdel	Remove a group from the system.
POST	/v1/admin/rotate	Rotate the JWT signing key in use.
GET	/v1/admin/system-conf	View current server configuration.



Table: Minioth public/admin API endpoints

Demonstration & Key Features

- Fully functioning web interface for:
 - Uploading datasets
 - Launching batch jobs with SQL, Python, or predefined tools
 - Viewing logs in real time (WebSocket streaming)
- Example: Executing DuckDB query on uploaded CSV → result saved and downloadable
- Job isolation: Every job runs in its own Kubernetes Pod



- **Accomplishments:**

- Designed and implemented a user-centric data platform on Kubernetes
- Combined identity, authorization, storage, and batch compute layers
- Delivered a fully working demo with real-world tools and users

- **Limitations:**

- Not expected to perform fast on really big data

- **Future Work:**

- Extend toolset: Jupyter, R, Julia, Tool For Tools
- Add CI/CD for job templates and data pipelines
- CLI tool, a road to the Shell
- Distributed orchestration...



Thank you!

kchalvatzis@tuc.gr

